Universidad del Valle de Guatemala Programación Orientada a Objetos
Segundo Ciclo, 2026
Catedrático: Kevin Antonio Velasquez Aguilar



Ejercicio 6 Polimorfismo vía Interfaces

Obed Natanael Tot Paiz
Carné:26971

Fecha de entrega: 31/10/2026

1. Análisis del problema

La plataforma se conecta con servicios externos de pagos, envíos y facturación electrónica y todas las integraciones tienen información común, pero sus capacidades son realmente diferentes.

Tipo de integración	**Consultar estado**	**Cobrar y reintegrar**	**Generar documentos**	**Registrar auditoría**
Pasarela de pago	Sí	Sí	No en este prototipo	Sí
Proveedor logístico	Sí	No	Sí: guía de envío	Sí
Certificador FEL	Sí	No	Sí: factura XML	Sí

Las interfaces permitirán incorporar proveedores con otras combinaciones de capacidades.

El sistema debe permitir:

- Cargar inicialmente al menos 10 integraciones de distinta naturaleza.
- Listar las integraciones activas y su estado de conexión.
- Buscar una integración por identificador y mostrar su configuración.
- Ordenar la lista por latencia promedio.
- Ejecutar comportamientos mediante contratos comunes.
- Ignorar un reintegro cuando la integración no admita operaciones financieras.
- Incorporar nuevos proveedores sin modificar la lógica central de checkout.

1.2. Clases e interfaces propuestas

Elemento	**Responsabilidad**
IntegracionAPI	Clase abstracta que reúne los datos y comportamientos comunes.
PasarelaPago	Representa una conexión con un proveedor de pagos.
ProveedorLogistico	Representa una conexión con un proveedor de envíos.
CertificadorFEL	Representa una conexión con un certificador de facturas.
Transaccional	Define las operaciones de cobro y reintegro.
Documentable	Define la generación de documentos.

Auditable	Define el registro de eventos.
CatalogoIntegraciones	Administra la única lista: carga, búsqueda y ordenamiento.
MotorCheckout	Ejecuta operaciones utilizando interfaces.
VistaConsola	Muestra el menú y obtiene los datos del administrador.
ControladorIntegraciones	Coordina la vista y el modelo.

1.3. Decisiones que justifican el diseño

Una sola lista polimórfica.

CatalogoIntegraciones almacena todos los objetos en ArrayList<IntegracionAPI>. No habrá listas separadas para pagos, logística y FEL. Para listar las activas, se recorre esa misma colección y se comprueba isActiva().

La carga inicial puede crear 4 pasarelas de pago, 3 proveedores logísticos y 3 certificadores FEL, cada uno con identificador único y configuración propia.

xReintegros no compatibles.

reintegrarSiCompatible(...) verifica si la integración implementa Transaccional. Si la implementa, delega en reintegrar(Transaccional, ...); si no, devuelve un resultado como "IGNORADO: servicio no transaccional" y el proceso continúa.

Así, un proveedor logístico no necesita implementar un reintegro ficticio ni lanzar una excepción. La comprobación se hace por interfaz, no por nombres de clases concretas.

Polimorfismo mediante parámetros.
generarDocumento(Documentable servicio, ...) puede recibir tanto un proveedor logístico como un certificador FEL. Cada objeto produce su documento correspondiente. El motor no necesita conocer la clase concreta.

Overloading o sobrecarga.
El catálogo presenta dos métodos con el mismo nombre y distintos parámetros:

- buscar(String id): búsqueda por identificador.
- buscar(String nombre, String endpoint): búsqueda alternativa por configuración.

La segunda búsqueda es una operación auxiliar propuesta para mostrar sobrecarga; el menú obligatorio utilizará la primera.

Comparable.
IntegracionAPI implementa Comparable<IntegracionAPI>. Su compareTo() compara la latencia promedio en orden ascendente. El catálogo ordena **la misma lista**, sin crear otra colección de integraciones.

Encapsulación e identidad.
Los atributos son privados. El identificador no tiene setter porque define la identidad del objeto. equals() y hashCode() usan ese identificador, y agregar() rechaza identificadores repetidos. toString() presenta los datos comunes y cada subclase añade su configuración particular.

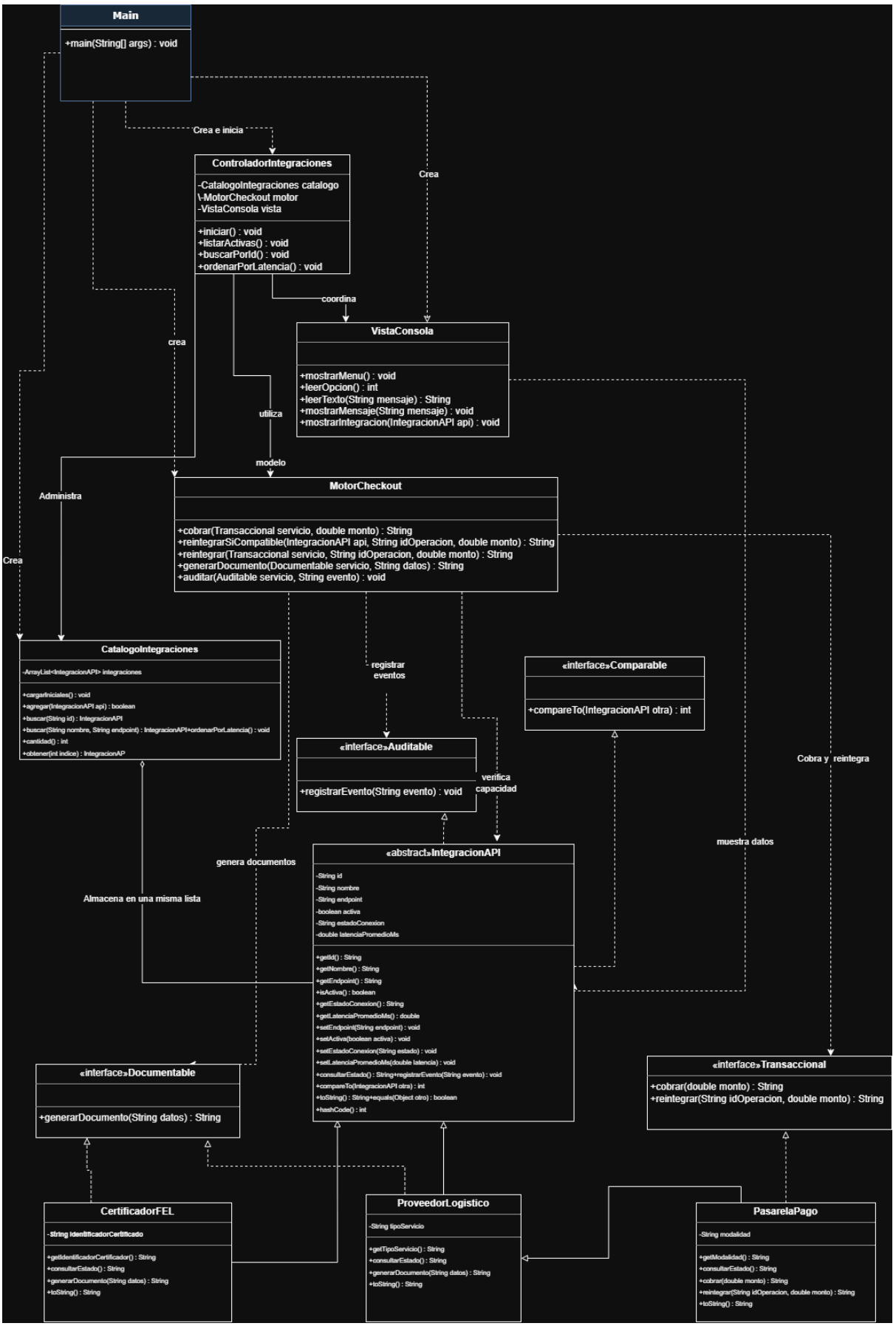
2. Diagrama UML

2.1. Que contiene el diagrama.

- Una sola lista: integraciones guarda objetos de las tres subclases.
- Interfaces: el motor recibe parámetros Transaccional, Documentable y Auditable.
- Reintegro incompatible: reintegrarSiCompatible() comprueba la interfaz Transaccional; si no está presente, devuelve un resultado de operación ignorada.
- Sobrecarga: los dos métodos buscar() tienen distintos parámetros.
- Ordenamiento: compareTo() compara la latencia promedio de menor a mayor.
- MVC: los grupos Modelo, Vista y Controlador indican dónde pertenece cada clase.

+ significa público y − privado. Se omiten los constructores para reducir el tamaño del diagrama.

2.2. Imagen del Diagrama


Main
+main(String[] args) : void
Crea e inicia
Crea
ControladorIntegraciones
-CatalogoIntegraciones catalogo
\-MotorCheckout motor
-VistaConsola vista
+iniciar() : void
+listarActivas() : void
+buscarPorId() : void
+ordenarPorLatencia() : void
coordina
VistaConsola
+mostrarMenu() : void
+leerOpcion() : int
+leerTexto(String mensaje) : String
+mostrarMensaje(String mensaje) : void
+mostrarIntegracion(IntegracionAPI api) : void
crea
utiliza
modelo
Administra
Crea
MotorCheckout
+cobrar(Transaccional servicio, double monto) : String
+reintegrarSiCompatible(IntegracionAPI api, String idOperacion, double monto) : String
+reintegrar(Transaccional servicio, String idOperacion, double monto) : String
+generarDocumento(Documentable servicio, String datos) : String
+auditar(Auditable servicio, String evento) : void
CatalogoIntegraciones
-ArrayList<IntegracionAPI> integraciones
+cargarIniciales() : void
+agregar(IntegracionAPI api) : boolean
+buscar(String id) : IntegracionAPI
+buscar(String nombre, String endpoint) : IntegracionAPI+ordenarPorLatencia() : void
+cantidad() : int
+obtener(int indice) : IntegracionAP
registrar eventos
«interface»Comparable
+compareTo(IntegracionAPI otra) : int
«interface»Auditable
+registrarEvento(String evento) : void
verifica capacidad
Cobra y reintegra
muestra datos
genera documentos
Almacena en una misma lista
«abstract»IntegracionAPI
-String id
-String nombre
-String endpoint
-boolean activa
-String estadoConexion
-double latenciaPromedioMs
+getId() : String
+getNombre() : String
+getEndpoint() : String
+isActiva() : boolean
+getEstadoConexion() : String
+getLatenciaPromedioMs() : double
+setEndpoint(String endpoint) : void
+setActiva(boolean activa) : void
+setEstadoConexion(String estado) : void
+setLatenciaPromedioMs(double latencia) : void
+consultarEstado() : String+registrarEvento(String evento) : void
+compareTo(IntegracionAPI otra) : int
+toString() : String+equals(Object otro) : boolean
+hashCode() : int
«interface»Documentable
+generarDocumento(String datos) : String
«interface»Transaccional
+cobrar(double monto) : String
+reintegrar(String idOperacion, double monto) : String
CertificadorFEL
-String IdentificadorCertificado
+getIdentificadorCertificador() : String
+consultarEstado() : String
+generarDocumento(String datos) : String
+toString() : String
ProveedorLogistico
-String tipoServicio
+getTipoServicio() : String
+consultarEstado() : String
+generarDocumento(String datos) : String
+toString() : String
PasarelaPago
-String modalidad
+getModalidad() : String
+consultarEstado() : String
+cobrar(double monto) : String
+reintegrar(String idOperacion, double monto) : String
+toString() : String

3. Enlace del Github

https://github.com/ntp90-cmd/Ejercicio-6-Polimorfismo-v-a-Interfaces-.git